

Ce que vous aurez à la fin

Une application que l'on consulte sans être administrateur : la liste des offres de stage, le détail d'une offre, la liste et le détail des entreprises, reliées entre elles, habillées d'une feuille de style, et **mesurées** — vous saurez combien de requêtes SQL coûte chacune de vos pages, et vous aurez divisé ce nombre par dix.

Point de départ

Votre dépôt de la séance précédente. Si vous l'avez manquée :

```
git switch -c tp3 tp3-depart
uv sync
uv run manage.py migrate
uv run manage.py loaddata demo
```

La dernière commande charge des entreprises, des offres, des étudiants et des candidatures. **Sans données, rien de ce TP n'a de sens** : une page vide coûte toujours une requête.

Si vous continuez sur votre propre dépôt, peuplez votre base avec votre `peuplement.py` de la semaine dernière, et vérifiez d'avoir au moins six offres réparties sur trois entreprises.

1. Une vue par fichier (15 min)

Transformez `stages/views.py` en package `stages/views/`, sur le modèle de `stages/models/`. Un fichier par sujet, pas un par fonction.

`stages/urls.py` ne doit pas changer d'une ligne : c'est le test que votre `__init__.py` fait son travail.

2. Trois pages neuves (60 min)

Voici ce qu'on doit pouvoir faire, une fois le serveur lancé. C'est votre seule spécification.

Depuis la page d'accueil, je vois **toutes les offres** : leur intitulé, l'entreprise qui les publie, les dates, le nombre de places et les compétences demandées. Je clique sur une offre et j'arrive sur **sa page**, avec sa description complète et un lien vers l'entreprise. De la page d'une **entreprise**, je vois son secteur, sa ville et la liste de ses offres, sur lesquelles je peux repartir. Et de n'importe où, je peux revenir aux offres ou aux entreprises.

Quatre contraintes, non négociables :

1. **Aucune adresse écrite en dur.** Tous les liens passent par `{% url %}`, donc toutes vos routes sont nommées.
2. **Une offre qui n'existe pas répond 404**, pas 500. Essayez `/offres/999999/` pour vérifier.
3. **Rien n'est dupliqué entre les pages.** L'en-tête, la navigation et le pied de page sont écrits une seule fois.
4. La liste des entreprises existante doit continuer de fonctionner, et passer elle aussi par le gabarit commun.

À vous les adresses, les noms de routes, la répartition des vues entre fichiers et la forme des templates. Committez au fur et à mesure, page par page.

3. Une feuille de style (20 min)

Ajoutez une feuille de style servie par Django et chargée avec `{% static %}`. Le but n'est pas de faire joli : une page lisible, un tableau qui se lit, des liens qu'on distingue du texte. Trente lignes suffisent, et le **noir et blanc doit rester lisible** — ces pages seront imprimées un jour.

- a. Où avez-vous placé le fichier, et pourquoi à cet endroit précis ?
- b. Pourquoi écrire `{% static 'stages/style.css' %}` plutôt que `/static/stages/style.css` ? Il existe au moins une bonne raison qui ne se voit qu'à la mise en production.

4. Compter les requêtes (35 min)

C'est la partie qui compte le plus aujourd'hui. Vous allez mesurer, corriger, puis **remesurer**.

4.1 La mesure

Dans un `run manage.py shell` :

```
from django.db import connection, reset_queries
from stages.models import Offre

reset_queries()
for offre in Offre.objects.all():
    offre.entreprise.nom
    list(offre.compétences.all())

len(connection.queries)
```

- a. Combien de requêtes ? Rapprochez ce nombre du nombre d'offres en base : quelle formule reconnaissez-vous ?
- b. Regardez `connection.queries[-1]`. Que contient-il, et que vous apprend-il sur ce que fait réellement l'ORM ?
- c. Comptez vos offres. Combien de requêtes avec deux cents offres ? Et avec deux mille ?

4.2 La correction

Corrigez la requête de votre vue avec `select_related` et `prefetch_related`, puis remesurez de la même manière — d'abord avec `select_related` seul, ensuite avec les deux.

Reportez vos trois nombres dans un fichier `mesures.md`, avec une phrase expliquant ce que chaque outil a supprimé :

mesure	requêtes
sans rien	
avec <code>select_related</code>	
avec les deux	

- d. Pourquoi faut-il deux outils différents ? Qu'est-ce qui distingue les deux relations que votre page suit ?
- e. `prefetch_related` **ajoute** une requête plutôt que d'en supprimer une. Expliquez pourquoi c'est quand même un gain.

4.3 Le piège

Votre page de détail d'une entreprise affiche la liste de ses offres. Mesurez-la aussi.

f. Souffre-t-elle du même défaut ? Si oui, corrigez-la ; sinon, expliquez pourquoi elle y échappe.

5. Une panne à diagnostiquer (15 min)

Dans un de vos templates, changez le nom d'une route dans un `{% url %}` — une lettre suffit. Rechargez la page.

a. Quel est le nom de l'erreur, et à quel moment survient-elle : au démarrage du serveur, au chargement de la page, ou au clic sur le lien ?

b. Comparez avec ce qui se serait passé si l'adresse avait été écrite en dur dans le template. Laquelle des deux erreurs préférez-vous découvrir, et pourquoi ?

Rétablissez le bon nom avant de committer.

À rendre

Poussez votre dépôt, avec `mesures.md` et un historique où l'on voit l'ordre dans lequel vous avez travaillé.

Restitution (20 min)

Sans machine.

1. Une page affiche cinquante offres et déclenche cent une requêtes. Expliquez d'où vient ce nombre à quelqu'un qui n'a jamais entendu parler du N+1.

2. `select_related` et `prefetch_related` ne s'emploient pas dans les mêmes cas. Donnez la règle qui permet de choisir, et un exemple de chaque tiré de votre propre projet.

3. Qu'est-ce qu'on perd, concrètement, à écrire `/offres/12/` en dur dans un template ? Donnez deux conséquences distinctes.
